Embedded Systems and IOT
2025/2026

Lecture 4

Assis. Prof. Dr. Elmahdy Maree

Embedded Systems and IOT

Coding

Cross compilation process

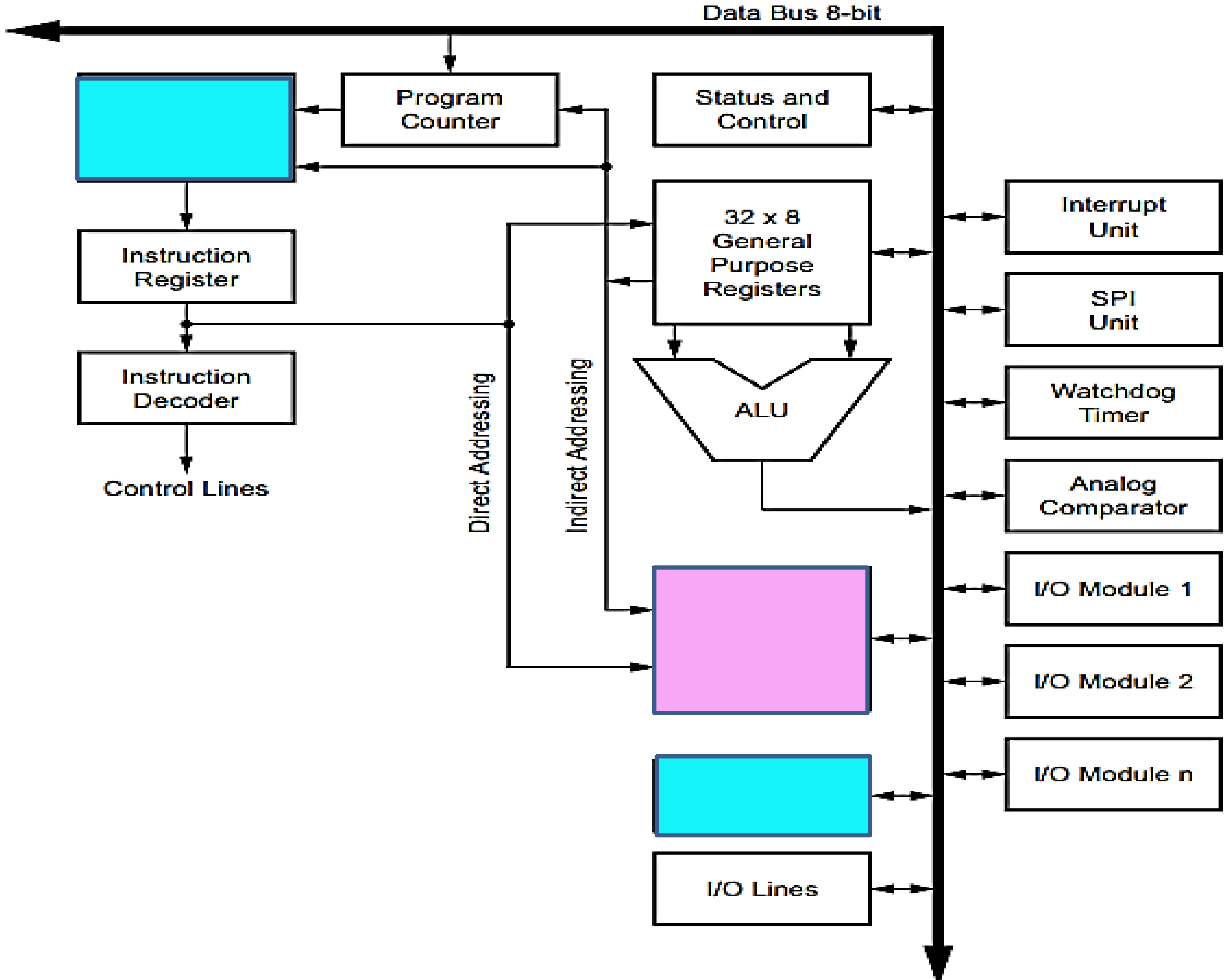
The technique of creating software for one type of processor on a different type of system is referred to as **cross-compiling**, and it is really the only way to create compiled software for microcontroller targets.

Small microcontrollers like the AVR (or any small 8-bit device, for that matter) simply don't have the resources to compile and link something like a C or C++ program. A more capable machine with resources such as a fast CPU, large-capacity disk drives, and lots of memory is used to do the compiling and linking, and then the finished program is transferred to the target for execution

Embedded Systems and IOT

CH1: AVR microcontroller block diagram

Block Diagram of the AVR Architecture



Embedded Systems and IOT

Introduction: ARM microcontroller block diagram

The flash memory is used to store program code, the SRAM is used to hold transient data such as program variables and the stack.

EEPROM can hold data that needs to persist between software changes and power cycles.

The flash and EEPROM can be loaded externally, and both will retain their contents when the AVR is powered off.

The SRAM is volatile, and its contents will be lost when the AVR loses power. The heart of an AVR microcontroller is the 8-bit

CPU, but what makes it a truly useful microcontroller is the built-in peripheral functions integrated into the IC with the CPU logic.

The peripheral functions of an AVR device vary from one type to another. Some have one timer; some have two or more (up to six for some types). Other parts may have a 10-bit A/D converter (ADC), whereas others feature a 12-bit converter. All AVR parts provide bidirectional I/O pins for discrete digital signals. Some versions also support a touchscreen and other types of interactive interfaces.

Embedded Systems and IOT

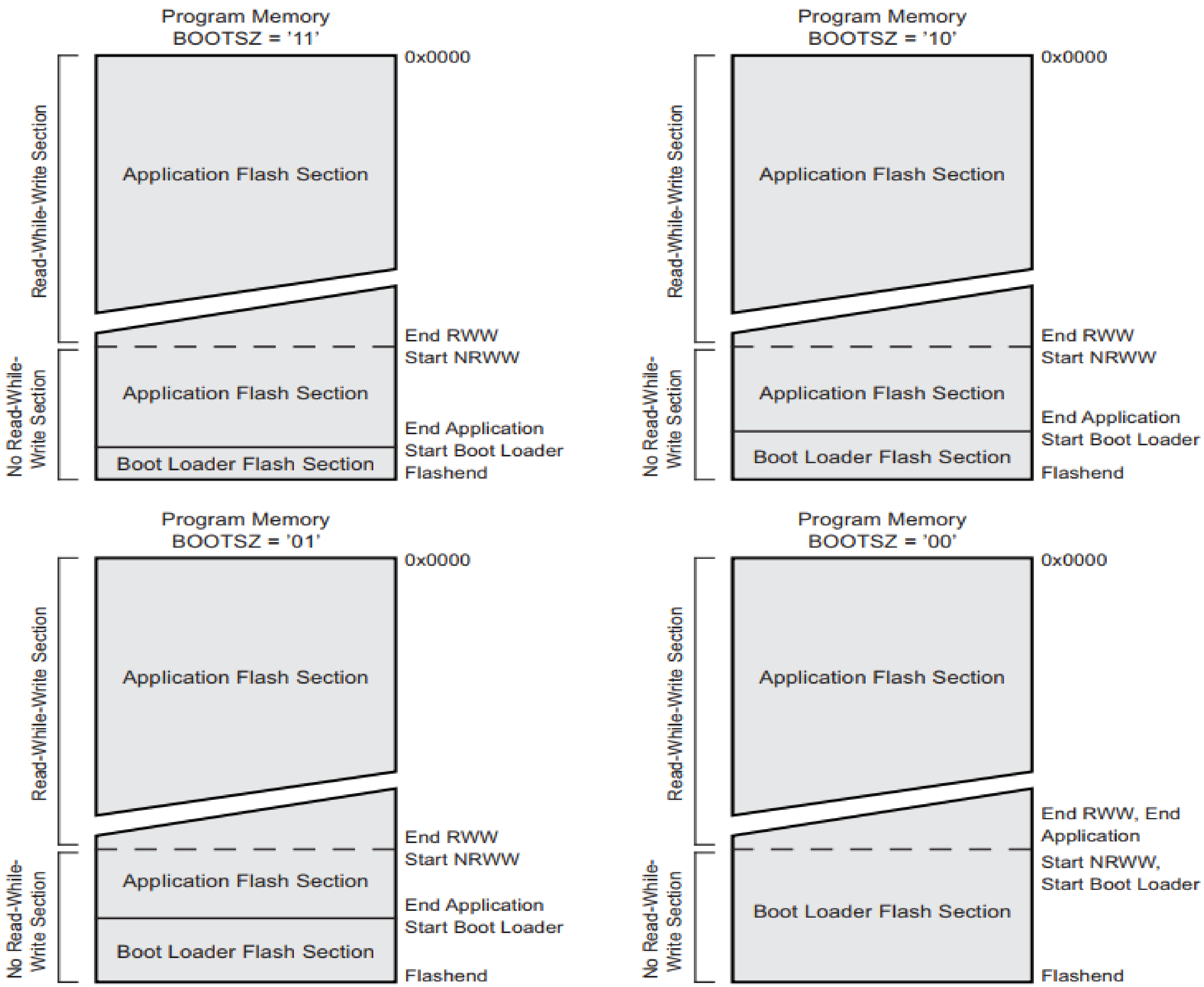
Coding

Bootloaders

The AVR family of microcontrollers provides reserved space in the on-board ***flash memory*** space for a bootloader. Once the MCU has been configured to use the bootloader, the address of this special memory space is the first place the AVR MCU will look for instructions when it is powered up (or rebooted). So long as the bootloader is not overwritten with an uploaded program, it will persist in memory between on-off power cycles.

The flash memory is used to store program code, the **SRAM** is used to **hold transient data** such as program variables and the stack.

Figure 26-2. Memory Sections



26.3.1 Application Section

The application section is the section of the flash that is used for storing the application code. The protection level for the application section can be selected by the application boot lock bits (boot lock bits 0), see Table 26-2 on page 232. The application section can never store any boot loader code since the SPM instruction is disabled when executed from the application section.

26.3.2 BLS – Boot Loader Section

While the application section is used for storing the application code, the The boot loader software must be located in the BLS since the SPM instruction can initiate a programming when executing from the BLS only. The SPM instruction can access the entire flash, including the BLS itself. The protection level for the boot loader section can be selected by the boot loader lock bits (boot lock bits 1), see Table 26-3 on page 232.

26.4 Read-While-Write and No Read-While-Write Flash Sections

Whether the CPU supports read-while-write or if the CPU is halted during a boot loader software update is dependent on which address that is being programmed. In addition to the two sections that are configurable by the BOOTSZ fuses as described above, the flash is also divided into two fixed sections, the read-while-write (RWW) section and the No read-while-write (NRWW) section. The limit between the RWW- and NRWW sections is given in Figure 26-2 on page 231. The main difference between the two sections is:

- When erasing or writing a page located inside the RWW section, the NRWW section can be read during the operation.
- When erasing or writing a page located inside the NRWW section, the CPU is halted during the entire operation.

Note that the user software can never read any code that is located inside the RWW section during a boot loader software operation. The syntax “read-while-write section” refers to which section that is being programmed (erased or written), not which section that actually is being read during a boot loader software update.

Table 27-6. Fuse High Byte for ATmega328P

High Fuse Byte	Bit No.	Description	Default Value
RSTDISBL[(1)]	7	External reset disable	1 (unprogrammed)
DWEN	6	debugWIRE enable	1 (unprogrammed)
SPIEN[(2)]	5	Enable serial program and data downloading	0 (programmed, SPI programming enabled)
WDTON[(3)]	4	Watchdog timer always On	1 (unprogrammed)
EESAVE	3	EEPROM memory is preserved through the chip erase	1 (unprogrammed), EEPROM not reserved
BOOTSZ1	2	Select boot size (see Table 26-7 on page 239 for details)	0 (programmed)[(4)]
BOOTSZ0	1	Select boot size (see Table 26-7 on page 239 for details)	0 (programmed)[(4)]
BOOTRST	0	Select reset vector	1 (unprogrammed)

Notes:
1. See Section 13.3.2 "Alternate Functions of Port C" on page 68 for description of RSTDISBL fuse.
2. The SPIEN fuse is not accessible in serial programming mode.
3. See Section 10.9.2 "WDTCSR – Watchdog Timer Control Register" on page 47 for details.
4. The default value of BOOTSZ[1:0] results in maximum boot size. See Section 27-11 "Pin Name Mapping" on page 246.

26.8.14 ATmega328P Boot Loader Parameters

In Table 26-7 through Table 26-9, the parameters used in the description of the self programming are given.

Table 26-7. Boot Size Configuration, ATmega328P

BOOTSZ1	BOOTSZ0	Boot Size	Pages	Application Flash Section	Boot Loader Flash Section	End Application Section	Boot Reset Address (Start Boot Loader Section)
1	1	256 words	4	0x0000 - 0x3EFF	0x3F00 - 0x3FFF	0x3EFF	0x3F00
1	0	512 words	8	0x0000 - 0x3DFF	0x3E00 - 0x3FFF	0x3DFF	0x3E00
0	1	1024 words	16	0x0000 - 0x3BFF	0x3C00 - 0x3FFF	0x3BFF	0x3C00
0	0	2048 words	32	0x0000 - 0x37FF	0x3800 - 0x3FFF	0x37FF	0x3800

Note: The different BOOTSZ fuse configurations are shown in Figure 26-2 on page 231.

27.5 Page Size

Table 27-9. No. of Words in a Page and No. of Pages in the Flash

Device	Flash Size	Page Size	PCWORD	No. of Pages	PCPAGE	PCMSB
ATmega328P	16K words (32K bytes)	64 words	PC[5:0]	256	PC[13:6]	13

Table 27-10. No. of Words in a Page and No. of Pages in the EEPROM

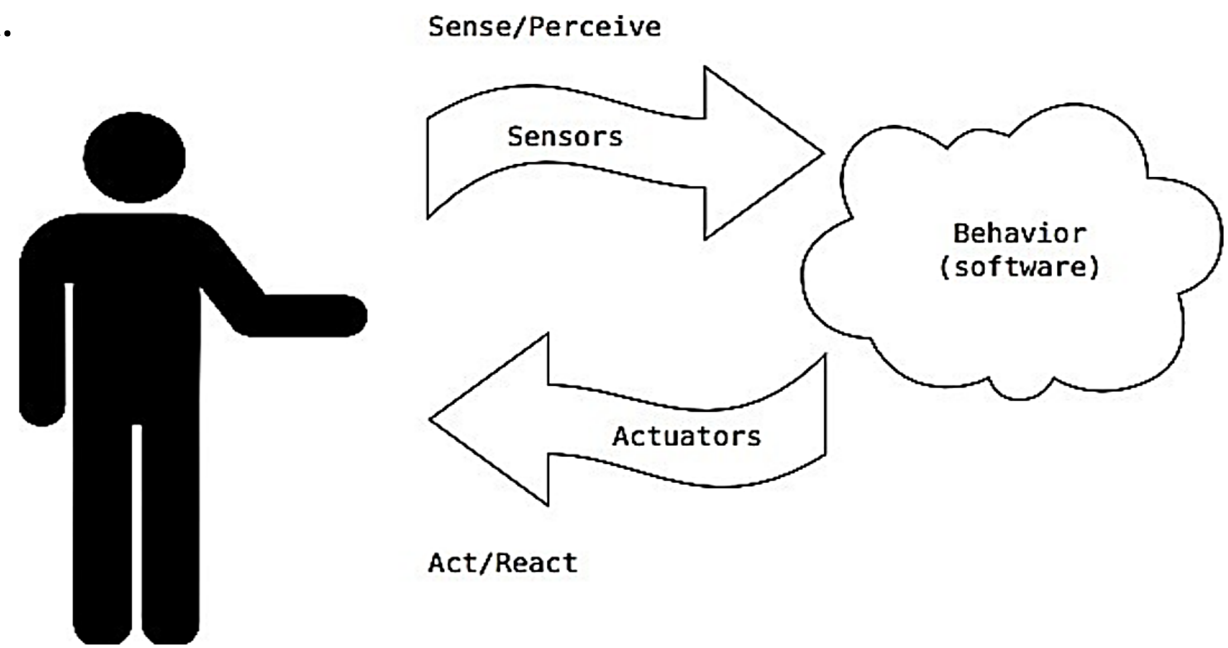
Device	EEPROM Size	Page Size	PCWORD	No. of Pages	PCPAGE	EEAMSB
ATmega328P	1K bytes	4 bytes	EEA[1:0]	256	EEA[9:2]	9

Embedded Systems and IOT

Coding

The Interactive Device

The Interactive Device is an electronic circuit that is able to sense the environment using sensors (electronic components that convert real-world measurements into electrical signals). The MCU processes the information it gets from the sensors with behavior that's implemented as software. The device will then be able to interact with the world using actuators, electronic components that can convert an electric signal into a physical action.

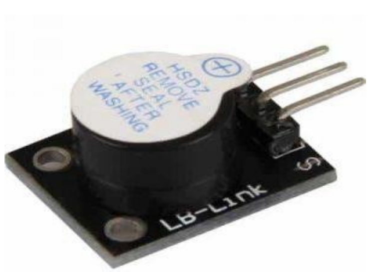
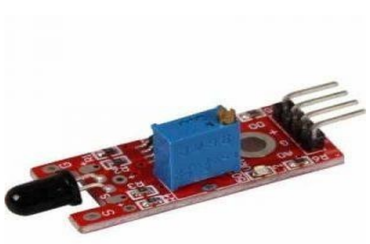


Embedded Systems and IOT

Coding

Sensors and Actuators

Sensors and actuators are electronic components that allow a piece of electronics to interact with the world. As the microcontroller is a very simple computer, it can process only electric signals. Once the sensors have been read, the MCU has the information needed to decide how to react. The decision-making process is handled by the microcontroller, and the reaction is performed by actuators.

Sensor/Module	Description
	**Active Piezo-Buzzer Module** This active piezo-buzzer module KY-012 will make a beep sound (2.5 Khz) when 3.3V is applied to its input pins. No PWM required. وحدة الصافرة الكهروضغطية النشطة تُصدر صوت صفير (بتردد 2.5 كيلوهرتز) عند تطبيق جهد 3.3 فولت على أطراف الإدخال الخاصة بها، ولا تحتاج إلى إشارة PWM للتشغيل.
	**Flame IR Sensor** The KY-026 IR Flame sensor module is sensitive the IR light spectrum emitted by open flames and triggers a digital output. مستشعر اللهب بالأشعة تحت الحمراء حساس لطيف الأشعة تحت الحمراء المنبعث من اللهب المفتوح، وتقوم بإطلاق إشارة رقمية عند اكتشافه.

Embedded Systems and IOT

Coding

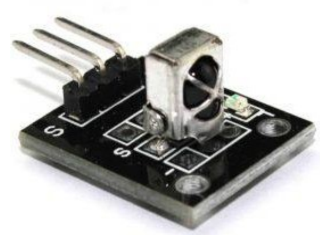
Sensors and Actuators

	**Heartbeat/Pulse Sensor** The KY-039 heartbeat detects heartbeat/pulse via the finger using a photoresistor and flashing an LED. Should *not* be used as a medical device. يكتشف نبضات القلب عبر الإصبع باستخدام مقاوم ضوئي (فوتورزيستور) وضوء LED يومض. ولا يُستخدم هذا المستشعر كجهاز طبي.
	**High Sensitivity Microphone Sensor** This is the KY-037 sensor is a highly sensitive module for picking up sound and audio. This sensor triggers a digital output based on the intensity of the sound. The trigger value can be adjusted based on the position of an on-board potentiometer. يُعد المستشعر KY-037 وحدة عالية الحساسية لالتقاط الأصوات والإشارات الصوتية. يقوم هذا المستشعر بإطلاق إشارة رقمية بناءً على شدة الصوت، ويمكن ضبط قيمة الاستشعار وفقًا لوضع المقاومة المتغيرة (البوتينشيومتر) الموجودة على اللوحة.
	**High-Voltage Relay Module** The KY-019 5V one channel relay module allows you to control high-voltage/high-current devices using a low-voltage digital output pin on the Arduino. يتيح الريلاى ذات القناة الواحدة (5 فولت) التحكم في الأجهزة ذات الجهد العالي أو التيار العالي باستخدام خرج رقمي منخفض الجهد من لوحة الأردوينو.

Embedded Systems and IOT

Coding

Sensors and Actuators



Infrared (IR) Sensor

Infrared IR sensor module (KY-022) for use in infrared data transmission projects. The perfect sensor for receiving button presses from an infrared remote control.

وحدة مستشعر الأشعة تحت الحمراء (KY-022) مخصصة للاستخدام في مشاريع نقل البيانات بالأشعة تحت الحمراء.وهو المستشعر المثالي لاستقبال إشارات ضغط الأزرار من جهاز تحكم عن بُعد يعمل بالأشعة تحت الحمراء.

Infrared (IR) Transmitter

The KY-005 Infrared transmitter module allows you to transmit IR signals and codes. This is great for remote controlling other devices or for home automation projects.

تتيح وحدة الإرسال بالأشعة تحت الحمراء KY-005 إرسال الإشارات والرموز بالأشعة تحت الحمراء.وهي مثالية للتحكم عن بُعد في الأجهزة الأخرى أو لاستخدامها في مشاريع أتمتة المنازل.

Joystick Module

This (KY-023) module is the perfect input device for gaming, controlling stepper motors, servos, and remote-control robotics projects. These have both analog (AO) and digital (DO) output options.

وحدة عصا التحكم تُعد وحدة جهاز إدخال مثالي للألعاب، والتحكم في محركات الخطوة Stepper Motors، والسيرفو (Servos)، ومشاريع الروبوتات التي يتم التحكم بها عن بُعد.توفر هذه الوحدة مخرجات تناظرية AO ورقمية DO

Embedded Systems and IOT

Coding

Sensors and Actuators

https://cpc.farnell.com/c/electronic-electrical-components/sensors-transducers/prl/results

MCS-136-1AR

SN36820

Data Sheet

RoHS

Proximity Switch & Magnet, Armoured, NO

COMUS / ASSEMTECH
• Metal magnetic contacts • Ideal for industrial or commercial use • Screws included • Standard 22AWG leads • 50 x 17 x 9.8mm (LxWxH)

LR03-L05M

SW03081

Data Sheet

RoHS

Float Switch, 5m Cable

TRITON CONTROLS

MCPIP-T12L-001

SN36994

Data Sheet

RoHS

Inductive Proximity Switch, M12, PNP, NO, Pre-wired 2m

MULTICOMP
The MCPIP-T12L-001 is a DC 3-wire Inductive Sensor with LED operation indicate lamp. Brass chrome plated, proof of oil, water acid, alkaline. Over-current protection. Widely applied in measuring, counting, Rpm measuring in mechanism, chemical, paper manufa...

03EN35T044(20/30)

SN35838

Data Sheet

RoHS

Thermal Switch, NO 30C

MICROTHERM

FS-10

SN36477

Data Sheet

RoHS

Flow Switch, 15mm, 0.5L/Min, 250V AC

TE CONNECTIVITY
FS-10 is a FS series Flow Switch in 15mm copper pipe. Suitable for mains water control, power shower, central heating systems, leak detection and cooling systems. • Minimal pressure drop • Operates from a small head of water • Vertical mount • Reed Switch ...

FS-05

SN36478

Data Sheet

RoHS

Flow Switch, Brass, AC

TE CONNECTIVITY
FS-05 is a FS series rugged brass liquid flow sensor with 22mm compression fitting for easy installation. Typical applications include mains water control, power shower, central heating systems, circulation pump protection and cooling systems. • Operates f...

SEN0368

SC21079

Data Sheet

RoHS

Non-contact Capacitive Liquid Level Sensor

DFROBOT
SEN0368 is a non-contact capacitive liquid level sensor for container OD<gt/>11mm. It is a non-contact liquid level sensor with status indicator and adjustable sensitivity. It can be used in liquid detection of non-metallic container or pipe (outer diamete...

MCS-136-2AR

SN36821

Data Sheet

RoHS

Proximity Switch & Magnet, Armoured, NC

COMUS / ASSEMTECH
• Metal magnetic contacts • Ideal for industrial or commercial use • Screws included • Standard 22AWG leads • 50 x

Embedded Systems and IOT

Coding

Sensors and Actuators

https://cpc.farnell.com/c/electronic-electrical-components/sensors-transducers/prl/results

1424684

CN26604

Data Sheet

RoHS

Sensor Connector, PROFINET CAT5, M12 Push-In, M12, Male, 4 Positions, Push In Pin

PHOENIX CONTACT

SERIES 8 A

SC10927

Data Sheet

Temperature Indicator Strip, 40°C/71°C, 10 Pack

THERMINDEX

• Irreversible temperature recording strip • Self-adhesive • Non-reversing-records peak temperatures • Water and oil resistant • 4 sensing areas • <lt/>1s response time • ±1°C accuracy • 50mm x 16mm (height x width)

PSG04177

SN36938

RoHS

3V to 5V Ultrasonic Distance Sensor, Front Header HC-SR04

MULTICOMP PRO

PSG04177 is an ultrasonic distance sensor. It has been engineered to work down to 3V making it absolutely ideal to connect directly to Micro Bit or Raspberry Pi GPIO pins using the 3.3V power. Works identically to the standard product although you will not...

LR03-L10M

SW03082

Data Sheet

RoHS

Float Switch, 10m Cable

TRITON CONTROLS

SEN0503

SC21091

Data Sheet

RoHS

IR Break Beam Sensor

DFROBOT

SEN0503 is an IR break beam sensor. This infrared break-beam sensor is a simple way to detect motion. This IR break beam sensor works up to 50cm with a fast and stable response of <2ms. With a small size of 20 x 10 x 9mm, the module can be conveniently ins...

PELR0222

LS02470

Data Sheet

RoHS

Piezo Buzzer, 12V dc, Leads

PRO SIGNAL

PELR0227

SN36934

Data Sheet

RoHS

12V Electro Mechanical Indicator Buzzer

PRO SIGNAL

• Operating Voltage: 7-16VDC • Rated Voltage: 12VDC • Rated Current: 23mA • Resonant Frequency: 400Hz • Sound Output (Min): 95dB • Lead Length: 150mm • Diameter: 26.4mm • Height: 17.6mm

MCS-137-1

SN36822

Data Sheet

RoHS

Proximity Switch, Magnet & Bracket, Armoured, NO

COMUS / ASSEMTECH

• Metal magnetic contacts • Ideal for securing chain link fence • Weather resistant Aluminium housing • Bracket included • 76.2 x 25.4 x 2.7 (LxWxH)

Embedded Systems and IOT

Report

List and describe 50 Sensors and Actuators

Embedded Systems and IOT

Coding

The Five Program Steps

When you first start to design a program, you should think of that program in terms of the following Five Program Steps.

1. Initialization Step

The purpose of the Initialization step is to establish the environment in which the program will run.

2. Input Step

The purpose of the Input step is to take the information provided by the sensors and buttons.

3. Process Step

The purpose of the process step is to determine which action should be taken according to the inputs and current state of the system.

4. Output Step

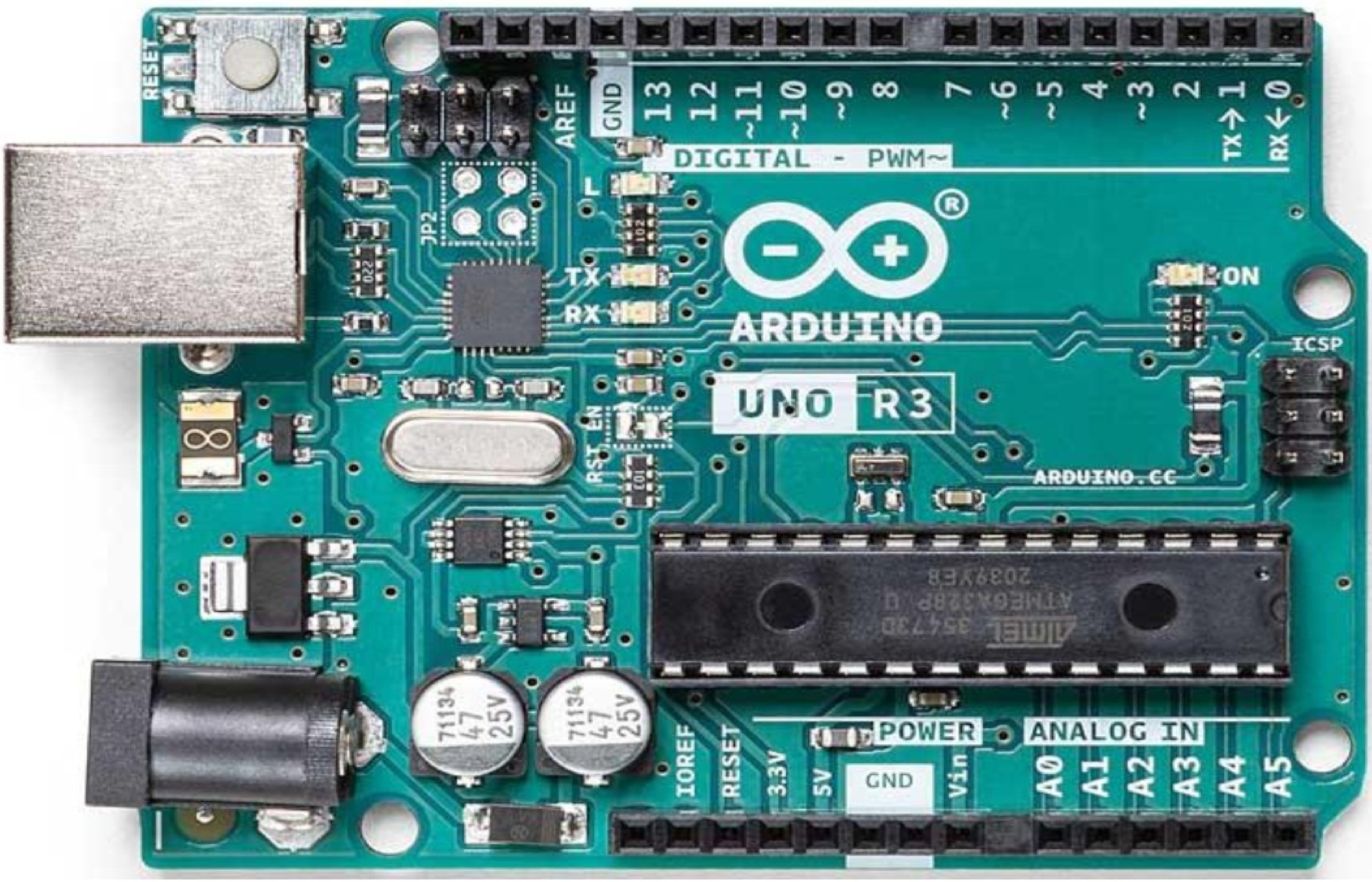
The purpose of the Output step is to provide out put devices such as display and actuators with the required information.

5. Termination Step

The purpose of the termination step is used to define the termination Process.

Embedded Systems and IOT

Arduino UNO R3 board



Embedded Systems and IOT

Coding

The setup() Function

The setup() function is called once when you reset the program. The purpose of the setup() function is to establish the environment in which your program will run.

```
// The assignment operator initializes an integer variable named led value to 13.

            int led = 13;

// the setup routine runs once when you press reset.
   void setup()
   {
       // initialize the digital pin as an output.
            pinMode (led, OUTPUT);
   }
```

In this case, the Initialization Step define pin 13 (the value of which is stored in led) of the digital **I/O pins** on the board to the output mode. It establishes this mode by calling a prewritten function named ***pinMode***() and passing the information that the function needs to perform its job as arguments to the function.

Embedded Systems and IOT

Coding

The loop() Function

After the setup() function completes its work, every Arduino C program automatically calls the loop() function.

```
void loop()
 {
   {
       digitalWrite(led, HIGH);     // turn the LED on (HIGH is the voltage level)
         delay(1000);               // wait for a second
       digitalWrite(led, LOW);      // turn the LED off by making the voltage LOW
         delay(1000);               // wait for a second
   }
 }
```

This sequence repeats until power is removed from the circuit, which means the **LED** simply sits there and blinks until the power is removed from the **MC** board or a component fails.

Embedded Systems and IOT

Project 1

SMART ALARM SYSTEM BASED ON INFRARED

IR based security alarm circuit with Arduino can detect any movement by adding two IR sensors. This Arduino controlled door alarm system can be installed near the door to identify anyone present at the door.

Required Components

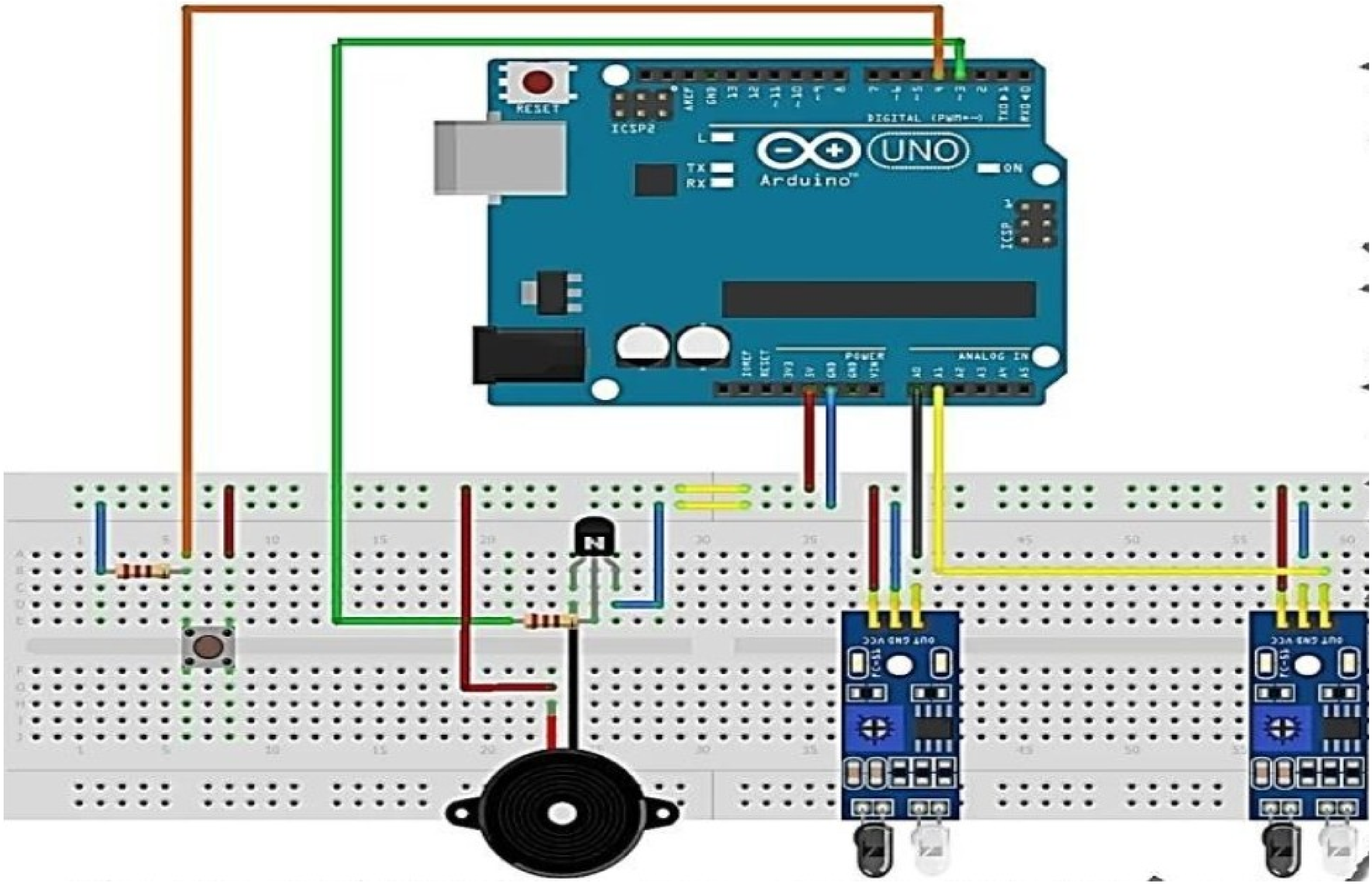
- Breadboard
- IR sensors
- Buzzer
- Arduino board (any model)
- Pushbutton
- NPN transistor
- 1K ohm resistor
- Jumper Wires
- USB cable for Arduino or 12v, 1A adapter.

Embedded Systems and IOT

Project 1

SMART ALARM SYSTEM BASED ON INFRARED

Circuit Diagram:



Embedded Systems and IOT

Project 1

```
    const int buzzer=3;                        // pin 3 is the buzzer output
    const int pushbutton=4;                    // pin 4 is the pushbutton input

void setup() {
    Serial.begin(9600);
    pinMode(buzzer,OUTPUT);                    //configure pin 3 as OUTPUT
    pinMode(pushbutton,INPUT);                 //configure pin 4 as INPUT }

void loop() {

    int sensor1_value = analogRead(A0);        //read the output of both sensors and compare to the threshold value.
    int sensor2_value = analogRead(A1);

if (sensor1_value >100||sensor2_value >100)
   {
     while(true)
      {
        digitalWrite(buzzer,High);             //sets the alarm ON
        if(digitalRead(pushbutton)==HIGH)
                         break;
      }
   }
else
              digitalWrite(buzzer,LOW);        //turn off alarm
}
```

يتم قراءة قيمة الإشارات **Analog** من الحساس الأول **A0** والحساس الثاني **A1**.

القيم تتراوح بين **0** إلى **1023** (حيث 0 = 0 فولت و1023 = 5 فولت).

Embedded Systems and IOT

Project 2

SMOKE DETECTION SYSTEM

We are going to make a Smoke Detector Alarm using Arduino. When the MQ2 Gas Sensor will detect the smoke level high, the red led will glow and the buzzer will start.

Required Components

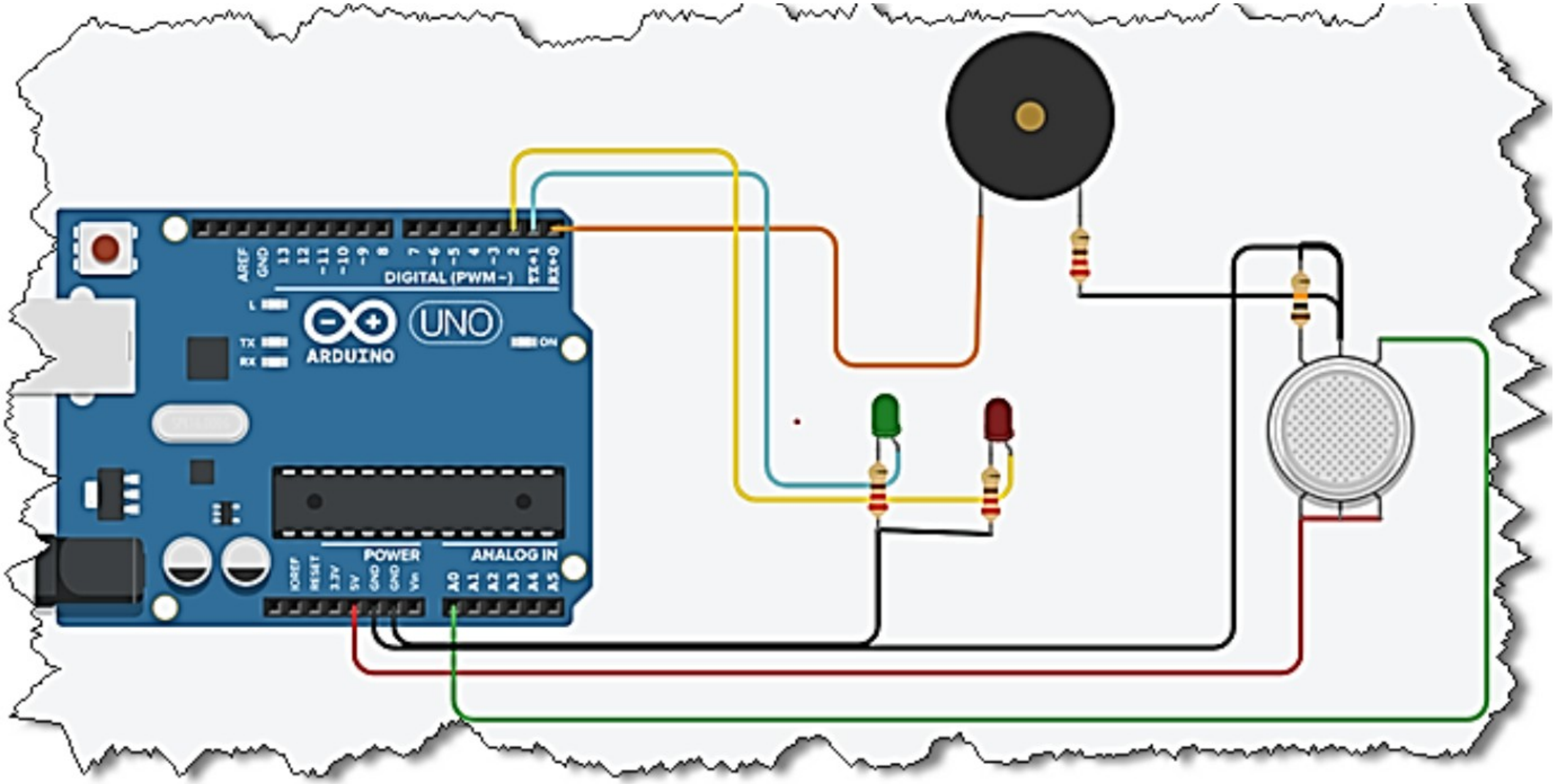
- Arduino Uno
- MQ2 Gas Sensor
- Buzzer
- 5V LED
- 100ohm Resistor
- Jumper Wires

Embedded Systems and IOT

Project 2

SMOKE DETECTION SYSTEM

Circuit Diagram:



Embedded Systems and IOT

Project 2 Code

```
//stored pins in variables
    #define gasSensor A0
    #define buzzer 0
    #define ledGreen 1
    #define ledRed 2
    #define THRESHOLD  600
void setup()
 {
        //Initialising all pins
            pinMode(gasSensor, INPUT);
            pinMode(buzzer, OUTPUT);
            pinMode(ledGreen, OUTPUT);
            pinMode(ledRed, OUTPUT);
}
```

Embedded Systems and IOT

Project 2 Code

```
void loop()
{
//Read data from the sensor
        int gas_value = analogRead(gasSensor);
//check data from sensor if there is smoke, if will execute otherwise else will execute
        if(gas_value > THRESHOLD)
        {      //          الزمن -التردد
            tone(buzzer,1000,500);
            digitalWrite(ledRed, HIGH );
            digitalWrite(ledGreen,LOW);
        }
        else
        {
        noTone(buzzer);
        digitalWrite(ledGreen, HIGH );
        digitalWrite(ledRed, LOW);
        }
    delay(200);
}
```

القيم تتراوح بين **0** إلى **1023** (حيث 0 = 0 فولت و1023 = 5 فولت).

تأخير بسيط مقداره 200 ملّي ثانية بين كل قراءة وأخرى لتخفيف سرعة التنفيذ.

Embedded Systems and IOT

Project 3

OBJECT TRACKING BASED ON ULTRASONIC

Ultrasound ranging is a complicated task that made easy by the readily available and inexpensive module for Arduino. To detect or measure the distance, it transmits the signal to the target and the target reflects that back. Arduino measure time is taken for complete travel. As the speed of sound is known the distance between module and target can be calculated easily.

Required Components

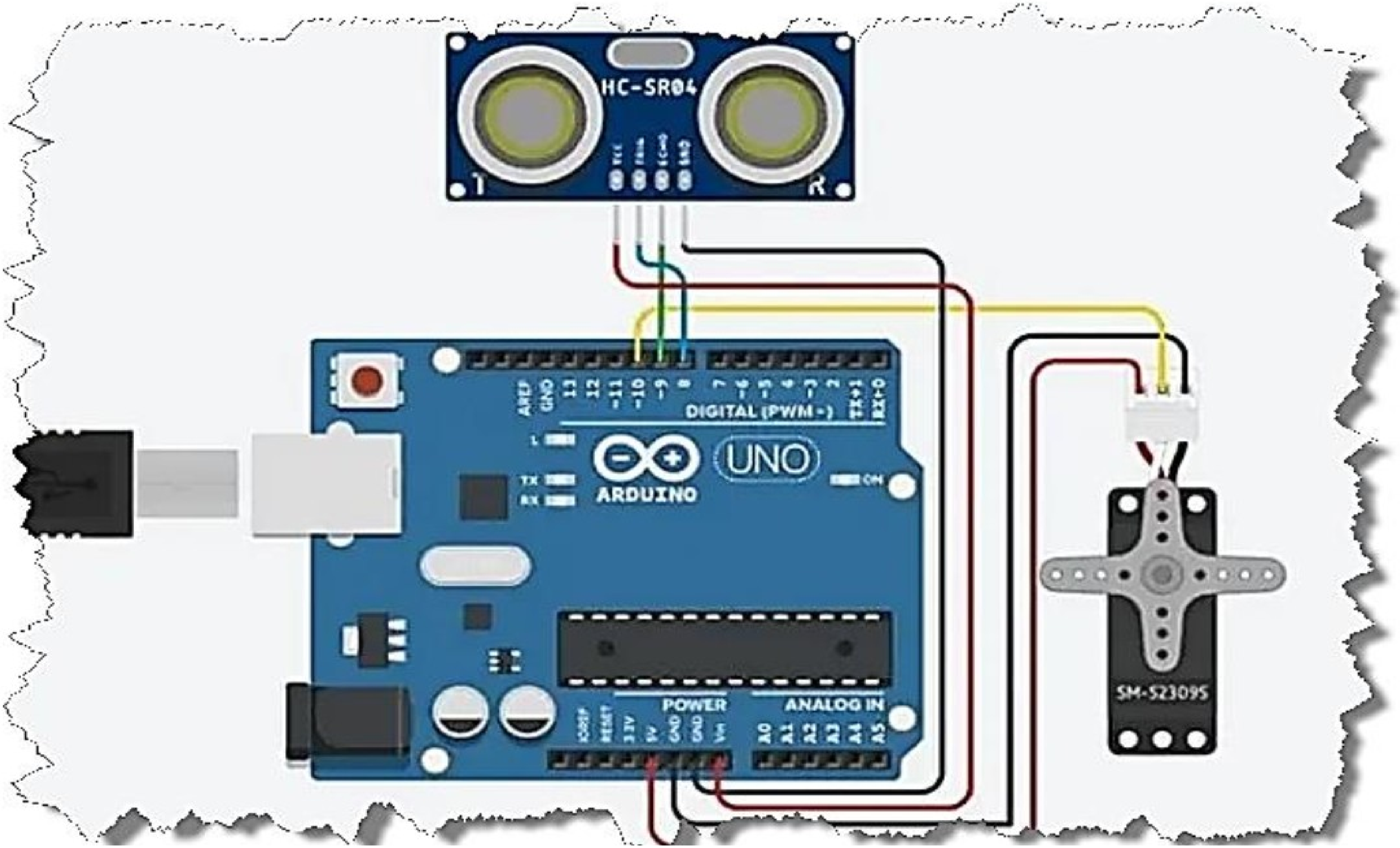
- **Arduino Uno**
- **SG90 Micro Servo**
- **Ultrasonic Sensor**
- **Wires**
- **Breadboard**

Embedded Systems and IOT

Project 3

OBJECT TRACKING BASED ON ULTRASONIC

Circuit Diagram:



Embedded Systems and IOT

Project 3

OBJECT TRACKING BASED ON ULTRASONIC

Code:

```
#include<Servo.h>
const int TriggerPin = 8;
const int EchoPin = 9;
const int motorSignalPin = 10;
const int startingAngle = 90;
const int minimumAngle = 6;
const int maximumAngle = 175;
const int rotationSpeed = 1;
Servo motor;
```

```
void setup(void)
{
pinMode(TriggerPin, OUTPUT);
pinMode(EchoPin, INPUT);
motor.attach(motorSignalPin);
Serial.begin(9600);
}
void loop(void)
{
static int motorAngle = startingAngle;
static int motorRotateAmount =
rotationSpeed;
motor.write(motorAngle);
```

Embedded Systems and IOT

Project 3

OBJECT TRACKING BASED ON ULTRASONIC

Code:

```
delay(10);
SerialOutput(motorAngle, CalculateDistance());
motorAngle += motorRotateAmount;
if(motorAngle <= minimumAngle || motorAngle >= maximumAngle) {
motorRotateAmount = -motorRotateAmount;
}}
int CalculateDistance(void)
{
digitalWrite(TriggerPin, HIGH);
delayMicroseconds(10);
digitalWrite(TriggerPin, LOW);
long duration = pulseIn(EchoPin, HIGH);
float distance = duration * 0.017F;
return int(distance);
}
```

Embedded Systems and IOT

Project 3

OBJECT TRACKING BASED ON ULTRASONIC

Code:

```
void SerialOutput(const int angle,
const int distance)
{
String angleString = String(angle);
String distanceString =
String(distance);
Serial.println(angleString + "," +
distanceString);
}
```

Questions

THANK YOU